Health Coach AI Agent

A Conversational, Evidence-Based Personal Wellness System

Ganesh Srivathsava Burri

vsganesh22@iitk.ac.in

Live System: health-coach-mvp-six.vercel.app

Repository: github.com/Ganapathi-007/Health-Coach-MVP

June 2026

Abstract

This report describes the design, architecture, and implementation of **Health Coach** — a 30-day personalized wellness coaching agent powered by Claude Sonnet 4.6. The system conducts daily conversational check-ins, maintains persistent memory across sessions, and delivers evidence-based coaching across six specialized health tracks. Unlike conventional chatbots that treat each interaction independently, Health Coach builds a living understanding of the user over time: it remembers what has been shared, recognizes behavioral patterns, and adapts its coaching accordingly. The core technical contributions are a multi-layer memory architecture combining synthesized long-term summaries with verbatim recent sessions, a track-specific safety guardrail system grounded in established clinical frameworks, and a conversational agent loop that replaces rigid question-answer forms with organically flowing coach-client dialogue.

Contents

1 Introduction **2**

2 System Architecture **2**

3 Technology Stack & Design Decisions **3**
3.1 Frontend — Next.js 15 . . . 3
3.2 Backend — FastAPI . . . 3
3.3 Database — Supabase JSONB . . . 3
3.4 AI Model — Claude Sonnet 4.6 . . . 3

4 Agentic Architecture **3**
4.1 The Problem with Form-Based Check-ins . . . 4
4.2 Conversational Agent Loop . . . 4
4.3 Memory Architecture . . . 4

5 Evidence-Based Track Design **5**
5.1 Track Routing . . . 5
5.2 Weekly Curriculum Structure . . . 5
5.3 Safety Guardrails . . . 5

6 Current System State **6**

7 Key Engineering Decisions **7**

8 Future Work **7**

9 Conclusion **8**

1 Introduction

Most AI wellness tools treat each conversation as a blank slate. The user explains their goals, answers a set of questions, receives generic advice, and starts over the next day. There is no accumulation of understanding, no recognition of patterns, and no sense of a relationship forming over time. This is not how effective coaching works.

A skilled human coach remembers. They recall what was said last week, notice when the same struggle keeps recurring, follow up on commitments made, and adjust their approach based on what they are learning about the person in front of them. The central design goal of Health Coach was to approximate this — not with a rigid rules engine, but with a language model that is given structured memory, evidence-based context, and constrained behavioral guidelines, and is then trusted to conduct a real conversation.

The system is fully deployed, accessible on both desktop and mobile, and has been tested through multiple days of real check-ins. This report documents the technical decisions behind it.

2 System Architecture

Health Coach follows a standard three-tier web architecture: a React frontend, a Python backend, and a PostgreSQL database. What makes it non-standard is the AI layer — a set of carefully constructed prompting pipelines that give Claude structured knowledge, persistent memory, and behavioral constraints on every call.

Table 1: System Components Overview

Layer	Technology	Role
Frontend	Next.js 15 (React)	Single-page app, chat UI, mobile responsive
Backend	FastAPI (Python)	API endpoints, prompt orchestration
Database	Supabase (PostgreSQL)	JSONB session storage, auth
AI Model	Claude Sonnet 4.6	Check-in conversations, coaching, memory synthesis
Auth	Supabase Auth	Email/password, JWT, password reset
Frontend Host	Vercel	CDN, auto-deploy from GitHub
Backend Host	Render	Managed Python hosting
Keep-Alive	cron-job.org	Pings `/health` every 5 min

3 Technology Stack & Design Decisions

3.1 Frontend — Next.js 15

Next.js was chosen for its zero-configuration Vercel deployment and server-side rendering capabilities. The entire application is a single React component with local state — no routing complexity, no Redux, no external state libraries. This was a deliberate MVP decision: keep the frontend simple, keep the complexity in the AI layer where it matters.

3.2 Backend — FastAPI

FastAPI's async-native architecture is well-suited to an AI application where each request involves one or more Claude API calls lasting 2–5 seconds. Pydantic models provide automatic request validation. The Anthropic Python SDK is first-class, making Claude integration clean and idiomatic. FastAPI's `BackgroundTasks` system is used for post-session memory updates — the user receives their coaching response immediately while the summary update runs concurrently behind the scenes.

3.3 Database — Supabase JSONB

The entire session state — user profile, check-in history, client summary, commitment history — is stored as a single JSONB document per user. This design choice was deliberate: the data model evolved significantly during development, and JSONB storage meant zero schema migrations. Supabase additionally provided a complete authentication system, eliminating the need to build signup, login, password reset, and JWT handling from scratch.

A critical production finding: Supabase's `Site URL` configuration must match the production domain, not `localhost`. When set to `localhost:3000`, the auth library enters an infinite token refresh retry loop on the production domain, causing multi-minute page load hangs.

3.4 AI Model — Claude Sonnet 4.6

Claude Sonnet 4.6 was selected over alternatives for three reasons. First, its instruction-following fidelity on complex multi-constraint prompts — critical when the system injects safety guardrails that must never be violated regardless of user input. Second, its conversational naturalism: the model can balance multiple simultaneous goals (respond to what was said, steer toward uncovered topics, decide when to end) without the response feeling mechanical. Third, the Anthropic Python SDK provides reliable structured output handling and clean async support.

4 Agentic Architecture

4.1 The Problem with Form-Based Check-ins

Early versions of the system generated 3–5 questions upfront and fired them sequentially. Users received coaching only after answering all questions. This approach had a fundamental flaw: it was indistinguishable from filling out a form. The coach never responded to what the user actually said — it moved to the next question regardless. If a user mentioned something significant, the system missed it entirely.

4.2 Conversational Agent Loop

The redesigned check-in uses two endpoints that together implement a genuine multi-turn conversation:

`POST /checkin/start` initializes the session and generates a personalized opening message. If the user made a commitment in the previous session, the opening names it explicitly and asks how it went. The opening is generated fresh each time — it references the user's specific history, not a template.

`POST /checkin/turn` is called on every user message. It receives the full conversation history as a proper message array, the week's topics, safety guardrails, and memory context. Claude generates the next coach message.

The key design decision is what Claude is told about when to end:

Listing 1: End-of-session instruction in coach turn system prompt

```
WHEN TO END:
- You decide: when topics are covered
  and the conversation feels naturally complete
- Close organically -- no "let's wrap up"
- Final message: specific coaching insight
  -> "For tomorrow:" commitment
  -> [END_SESSION] on the very last line
```

The `[END_SESSION]` token is stripped by the backend before the response reaches the frontend. No hard exchange limits are imposed — the conversation ends when a real coach would end it.

4.3 Memory Architecture

The most technically significant component is the memory system. Without persistent memory, the coach re-asks questions users have already answered — a failure that immediately breaks the illusion of a real coaching relationship.

The system uses a two-layer architecture:

Long-term: Client Summary. After each session closes, a background Claude call synthesizes the entire history into a structured client profile. This captures specific details — behavioral triggers, recurring patterns, commitments kept or broken — in the user's own words. No hard word limit is imposed; specificity is preserved over brevity. The summary grows organically across 30 days.

Short-term: Verbatim Recent Sessions. The last three completed sessions are passed in full alongside the summary. This preserves the precise language and detail of recent

exchanges — important because summarization inevitably loses nuance.

Both layers are injected into the system prompt via a helper function:

Listing 2: Memory context builder

```
def _build_memory_context(client_summary, previous_checkins):
    parts = []
    if client_summary:
        parts.append(f"CLIENT PROFILE:\n{client_summary}")
    if previous_checkins:
        recent = [c for c in previous_checkins
                  if c.user_responses][-3:]
        # Format last 3 sessions verbatim
        ...
    return "\n\nDo NOT re-ask anything already answered."
```

The total context injected remains well within Claude's 200K token window even at day 30 (estimated 20,000–40,000 tokens). The summary update runs as a FastAPI `BackgroundTask` — the final coaching response returns to the user immediately, and the summary update completes concurrently. This eliminated a 3–5 second delay on session close.

5 Evidence-Based Track Design

5.1 Track Routing

At onboarding, the user describes their health concerns in free text. A lightweight Claude call reads this and routes them to one of six tracks. The routing prompt uses a strict decision hierarchy to handle overlapping concerns — behavioral addiction patterns take priority over anxiety, for instance, since the coaching approach differs significantly.

5.2 Weekly Curriculum Structure

Each track follows a 4-week evidence-based curriculum. Every week specifies: a thematic name, a focus area, four specific techniques with their evidence citations, and the clinical framework being applied. This structure is injected into every coaching prompt as context — not as a rigid script, but as a set of topics the coach is expected to weave into the conversation naturally.

5.3 Safety Guardrails

Each track has a set of hard safety rules injected into every Claude call. These are constraints framed as non-negotiable in the system prompt. Examples:

- **Anxiety track:** Never diagnose GAD, panic disorder, PTSD, or any DSM condition. If suicidal ideation is mentioned, stop coaching immediately and provide the 988 Suicide and Crisis Lifeline.
- **Weight loss track:** Never set calorie targets or macronutrient ratios. If eating disorder patterns emerge, stop coaching and recommend specialist care.

Table 2: Program Tracks and Clinical Frameworks

Track	Target Concern	Evidence Base
Anxiety & Stress	Chronic stress, worry, burnout	MBSR (Kabat-Zinn), CBT (Beck Institute), ACT (Hayes)
Weight Loss	Metabolic health, habits	NIH/CDC DPP, Motivational Interviewing (Miller & Rollnick)
Skin Health	Acne, eczema, inflammation	Bowe & Logan gut-skin axis, IFM 5R Protocol
Energy & Sleep	Fatigue, insomnia, brain fog	CBT-I (Morin/Espie), Borbely 2-Process Model, Panda TRE
Behavioral Change	Compulsive habits, addiction	Duhigg habit loop, Marlatt relapse prevention, Brewer urge surfing
General Wellness	Mixed or undefined	ACLM 6 Pillars, Blue Zones, Fogg Tiny Habits

- **Energy track:** If post-exertional malaise is reported (fatigue worsening after activity), stop all exercise recommendations immediately — this pattern requires medical evaluation.
- **Behavioral track:** Never use shame language around lapses. Never say "back to square one." Shame is the primary documented driver of relapse in behavioral change literature.

6 Current System State

The following features are fully implemented and deployed:

- **Authentication** — signup, signin, password reset, persistent session across devices
- **Onboarding** — free-text intake parsed by Claude, automatic track routing, personalized welcome
- **Daily check-in** — conversational multi-turn agent with organic start and end, "I want to talk more" continuation
- **Persistent memory** — client summary updated after every session, verbatim recent sessions injected into every turn
- **Safety guardrails** — track-specific constraints on every Claude call
- **Progress analysis** — pattern detection across check-in history
- **Protocol Q&A** — answers grounded strictly in track-specific reference documents

- **4-week program roadmap** — visible to users with current week highlighted
- **Mobile responsive UI** — bottom navigation, full-width layout, iPhone safe-area handling

7 Key Engineering Decisions

Table 3: Architectural Decisions and Rationale

Decision	Rationale
JSONB session storage	Schema evolves constantly in early development; no migrations needed as new fields are added
Background task for memory update	Unblocks the final coaching response; user does not wait for a second Claude API call
`[END_SESSION]` token	Lets Claude decide organically when to close; eliminates arbitrary exchange count limits
No hard question limits	Previous "3 questions" and "7 exchange" ceilings were arbitrary boundaries that made the agent feel like a form
Verbatim recent sessions	Summarization loses specific detail; passing last 3 sessions in full preserves the nuance the coach needs
Track-specific protocols in code	Version-controlled with the codebase; easier to iterate than external documents
Regex JSON fallback	Larger system prompts occasionally cause Claude to wrap JSON in prose; `re.search` prevents production `JSONDecodeError`

8 Future Work

Goal drift detection. After several sessions, Claude analyses whether the user's actual concerns have shifted away from their assigned track. If consistent drift is detected, the user is offered a track switch.

Adaptive intensity. When a user is consistently struggling with a week's techniques, the coach dials back rather than pushing harder. Conversely, when a user is thriving, additional depth is introduced.

Safety screener at onboarding. PHQ-2 style screening questions before track assignment, with flags for ME/CFS patterns and eating disorder risk.

Evaluation framework. An LLM-as-judge eval script that simulates conversations across all tracks and weeks, scoring each against a rubric: topic coverage, safety compliance, memory usage, commitment quality, and conversational naturalness.

9 Conclusion

The central bet of this system is that the value of an AI health coach comes not from any single interaction, but from what accumulates across many interactions. A coach who forgets everything between sessions is not a coach — it is a form with a personality. Health Coach is built around the opposite principle: memory is a first-class feature, not an afterthought.

The technical choices made here — JSONB storage for schema flexibility, conversational agent loops without hard limits, a two-layer memory architecture, track-specific guardrails grounded in clinical literature — are all in service of that single goal: making the user feel, over time, that someone is paying attention.

Live deployment: health-coach-mvp-six.vercel.app
Source code: github.com/Ganapathi-007/Health-Coach-MVP